How AI Coding Actually Works

Tokenization, Context, Prompts, and Tools
Week 2 - Vibe Coding 101 | Instructor: Goker Ezberci

How AI Coding Actually Works

Tokenization, Context, Prompts, and Tools

Week 2 - Vibe Coding 101 | Instructor: Goker Ezberci

Agenda

By the end of Week 2 and 3, you will:

1. Understand tokenization, context windows, and next-token prediction
2. Analyze AI coding tools across categories and tradeoffs

   Next week;

3. Apply prompt engineering and context engineering techniques
4. Evaluate AI-generated code for quality, correctness, and maintainability

Agenda

By the end of Week 2 and 3, you will:

Week 2 & 3 Objectives

1. Understand tokenization, context windows, and next-token prediction
2. Analyze AI coding tools across categories and tradeoffs

→ Next week;

3. Apply prompt engineering and context engineering techniques
4. Evaluate AI-generated code for quality, correctness, and maintainability



Always be learning! (ABL)

Scan to vote

Learning Objectives

By the end of Week 2 and 3, you will:

Week 2 and 3 Objectives

- Understand tokenization, context windows, and next-token prediction
- Analyze AI coding tools across categories and tradeoffs

Next Week Objectives

- Apply prompt engineering and context engineering techniques
- Evaluate AI-generated code for quality, correctness, and maintainability

Why This Matters: The Problem

Without understanding the mechanics:

The Risks

- Surprised when same prompt gives different output
- Won't know when to trust AI vs. verify manually
- Miss optimization opportunities (context bloat, cost)
- Pick the wrong tool for the wrong job

Example: AI generated secure login endpoint

But actually: hashes password incorrectly, doesn't validate email, wrong API response format

⚠ The AI was confident. You didn't validate. This is why mechanics matter.

Part 1: LLM Fundamentals - Agenda

1. **Tokenization** — Why some words are 1 token, others are 4

2. **Context Windows** — What 200K tokens actually means

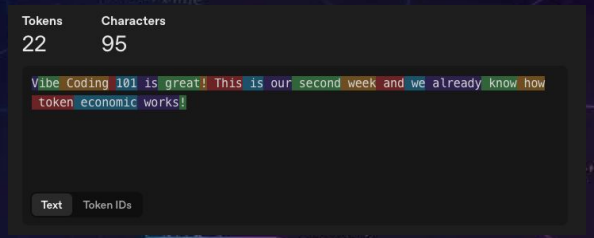
3. **Temperature & Sampling** — Why same prompt gives different code

4. **Next-Token Prediction** — The core mechanic (and why hallucinations happen)

5. **Strengths & Weaknesses** — Where AI excels and struggles

6. **Transformers** — Conceptual understanding of attention

Tokenization — The Foundation



A **token** = subword unit, NOT a word

XMLHttpRequest → 4 tokens: XML | Http | Request | "

Examples:

- "Function" → 1 token
- "**XMLHttpRequest**" → 4 tokens (XML/Http/Request)
- "**const myVar = 42;**" → 6 tokens

Function → 1 token

const | myVar | = | 42 | ; | "

Why it matters:

Cost: Charged per token. Verbose context = higher bills

Context efficiency: Long context = fewer tokens for code

Model variance: Claude, GPT, Gemini tokenize differently

Action: Use a tokenizer tool to count tokens in your PRD

Action: Use a tokenizer tool to count tokens in your PRD

https://platform.openai.com/tokenizer

Context Windows — The AI's Memory

Context window = how many tokens the model can "see" at once

Current Models (Feb 2026):

- Claude 3.5 Sonnet: 200K tokens (industry standard)
- Gemini 2.5 Pro: 1M tokens (5x larger!)
- GPT-4: 128K tokens
- Local models: 8K–32K tokens

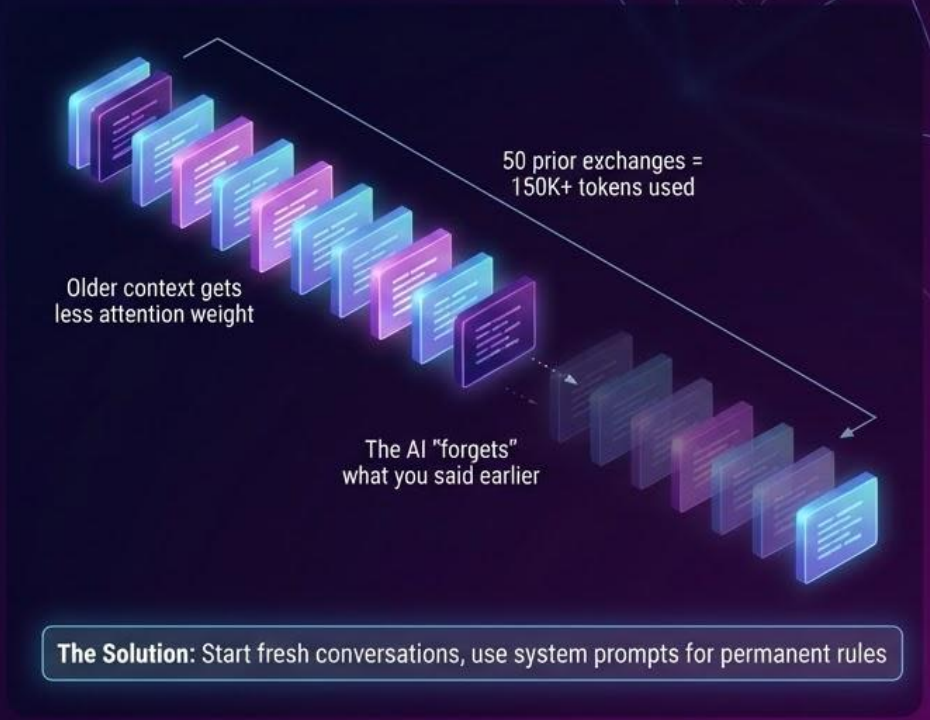
The Problem:

After 50 prior exchanges in conversation = 150K+ tokens used

Older context gets less attention weight

The AI "forgets" what you said earlier

Solution: Start fresh conversations, use system prompts for permanent rules



Context Windows — The AI's Memory

Context window = how many tokens the model can "see" at once

CURRENT LANDSCAPE — FEBRUARY 2026

Model	Context	Notes
Gemini 3 Pro	10M tokens	Largest available — entire codebases in one shot
Llama 4 Scout	10M tokens	Open-source! MoE 17B active / 109B total
Gemini 2.5 Pro	1M tokens	Production workhorse, 2M coming soon
Claude Opus 4.6	1M tokens (beta)	NEW TODAY! 128K output, agent teams, adaptive thinking
GPT-5.2	400K tokens	128K output, 3 modes: Instant / Thinking / Pro
Claude Sonnet 4.5	200K (1M beta)	Best recall consistency (<5% degradation at full ctx)
DeepSeek R1	164K tokens	$0.55/M tokens — best budget reasoning model
Local models (Ollama)	8K–128K	Varies by model; Llama 3.1 supports 128K

🚀 In Practical Terms

- **200K tokens** = 150,000 words = 300 pages
- **1M tokens** = 750,000 words = 5 novels
- **10M tokens** = entire monorepo + all docs
- For most projects, **200K is enough**. Bottleneck = attention, not capacity.

⚠️ The Problem

- System prompt: 1,000 tokens
- Your PRD: 3,000 tokens
- Chat history (50x): **150,000 tokens**
- Your new request: 1,000 tokens
- Remaining output: **45,000 tokens**
- Older context crowded out. AI "forgets."

Solution: Start fresh conversations for new features. Use system prompts & CLAUDE.md for permanent rules.

Why AI “Forgets”

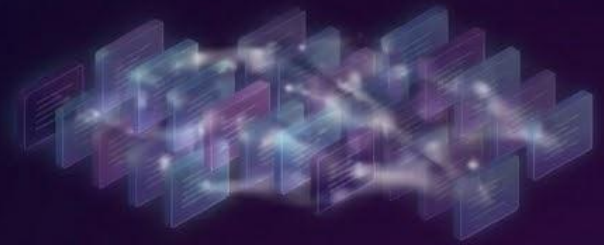
The transformer has fixed-size attention weights.

Attention Distribution Across Context:

The Fix:

- Start fresh conversations for different topics
- Use system prompts for permanent rules
- Keep context focused (one conversation per feature)

✅ **GOOD:** One conversation per feature



❌ **BAD:** 200-message conversation trying to build 5 features

Temperature & Sampling

Temperature = how random the model is

Temperature Scale:

0.0
Deterministic (always same output)

0.3
Consistent (minor variations)

0.7
Balanced (some creativity, mostly predictable)

1.0
Creative (diverse outputs)

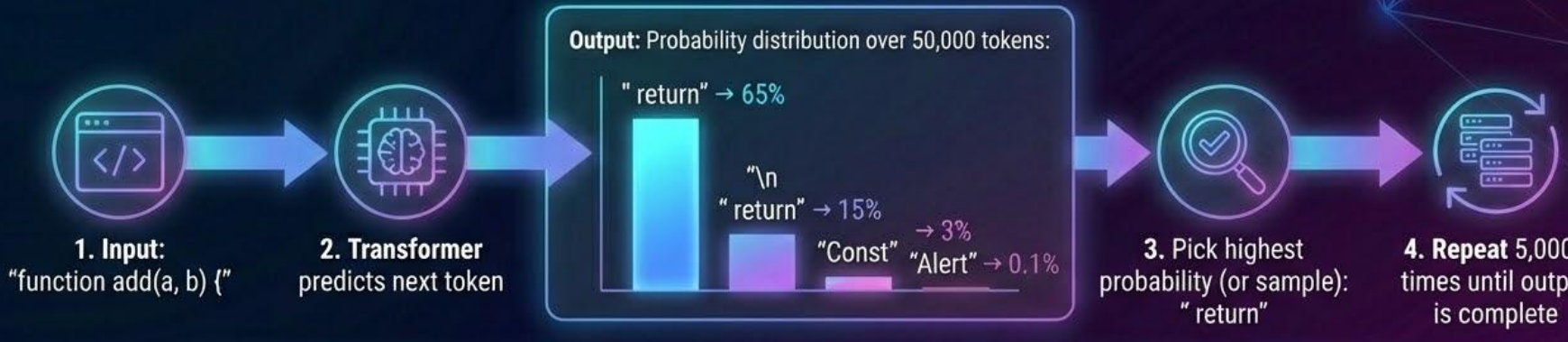
1.5+
Chaotic (wild, unpredictable)

When to Use:

- Production code: 0.0–0.3 (consistency, reproducibility)
- Design/brainstorming: 0.7–1.0 (creativity without chaos)
- Exploration: 1.0+ (wild ideas, accept lower quality)

Next-Token Prediction — How Code Gets Generated

An LLM **doesn't think**. It predicts the next token repeatedly.



The Result & Key Insight:

Result: Code that looks coherent because training data includes lots of working code. This is fundamental to how LLMs work.

Why Hallucinations Are Mechanical

Hallucinations aren't bugs. They're features of next-token prediction.

Why it happens:

```
encryptionKey: user.encryptionKey // ← Hallucination!
```

Model saw:
db queries return user objects

Training data includes:
some objects have encryption keys

Generated:
this field probably exists

Confidence level:
HIGH (pattern is plausible)

The Real Problem:
The model has no "I'm not sure" mechanism. It's always confident.

How to Defend:

1. Know your domain → Spot hallucinations

2. Always test
→ Code will crash if field doesn't exist

3. Validate APIs
→ Check official docs

4. Pair programming
→ Have humans review AI output

Where AI Excels (Pattern Tasks)

AI is GREAT at:


Boilerplate (React components, API handlers, CRUD endpoints)


Code transformation (refactoring, linting, formatting)


Generating from specifications (test cases from code)


Documentation (docstrings from function signatures)


Language translation (Python ↔ JavaScript)


Fixing syntax errors


Explaining code

Why? These are high-pattern tasks.


Training data includes thousands of examples.


KEY INSIGHT:

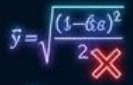
AI codegen tools are basically very expensive **syntax highlighting + autocomplete**.

But that's incredibly valuable for productivity.

Where AI Struggles (Reasoning Tasks)

AI STRUGGLES with:


Novel algorithms (not in training data)


Complex math (requires symbolic reasoning)


Long chains of logic (attention dilutes)


Security analysis (requires global perspective)


Performance optimization (hard without measurement)


Debugging esoteric issues (needs deep context)


Guarantees & proofs

Rule of Thumb:

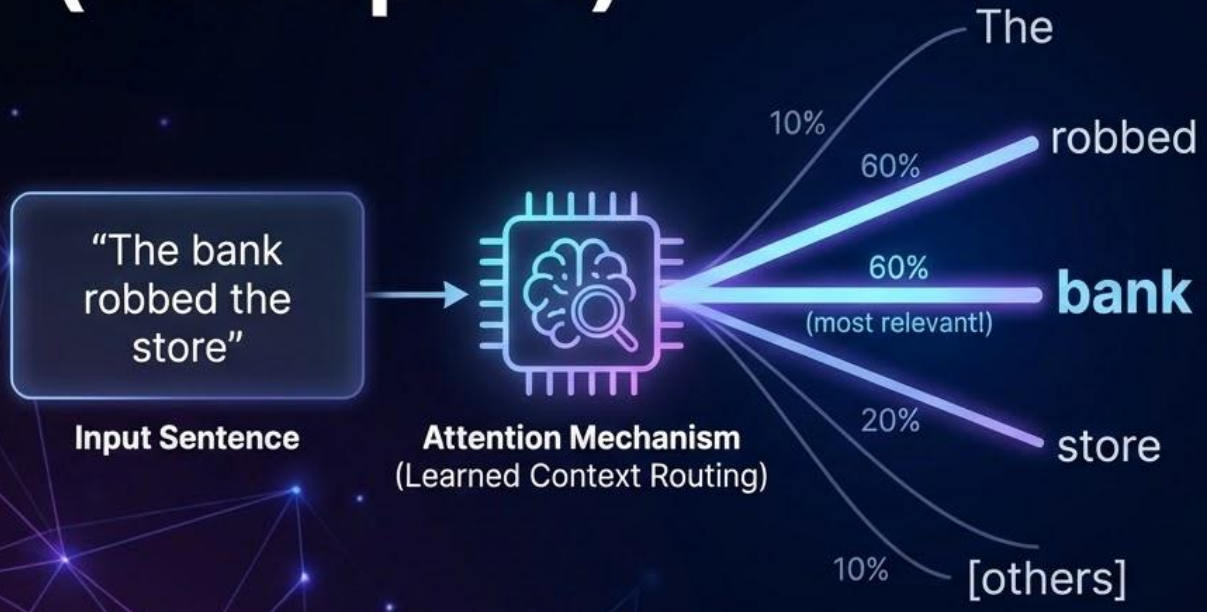

USE AI FOR: “Generate code from a pattern I provide”


DON’T USE AI FOR: “Solve this hard problem for me”

KEY INSIGHT:

AI is not a replacement for engineering intuition. It’s a tool for implementing intuitive decisions quickly.

The Transformer Architecture (Conceptual)



Why STRUCTURE HELPS:

- ✅ **Headers** → Attention weight
- ✅ **Bullet points** → Attention weight
- ✅ **Code examples** → Attention weight
- ❌ **Random prose** → Distributed, weak attention

Better formatted prompts = Better attention routing = Better results

- Transformers by 3Blue1Brown - Youtube
- LLM Visuazliation

Part 2 – AI Tools Landscape

The tool landscape is moving FAST. By 2027, different tools will lead.
But the **CATEGORIES** are stable:

1. IDE-native:
Cursor, Windsurf, GitHub Copilot

Tools integrated into your code editor

2. Terminal-native:
Claude Code, Gemini CLI, aider

Tools you run from the command line

3. Rapid prototyping:
Bolt.new, v0, Lovable, Replit

Full-stack platforms for building quickly

REMEMBER: Tools change every 3 months. Disciplines last decades.
Learn the principles, not just the tools.

IDE-Native Tools Deep Dive

CURSOR
(VS Code fork, 200K context)

- **.cursorrules** file for project standards
- **Composer**: multi-file editing
- **Tab completions**
- **Cost:** $20/month
- **Best for:** Teams wanting IDE-integrated AI

WINDSURF
(New, 200K context)

- **Cascade agent**: multi-step problem solving
- **Tab v2**: Released Feb 3, 2026 (10x speed claims)
- **Agentic by design** (writes code + runs terminal)
- **Cost:** $40/month
- **Best for:** Complex refactoring, multi-file changes

GITHUB COPILOT
(Enterprise standard, 128K context)

- **Claude** and **Codex** agents
- **Enterprise admin** controls
- **JetBrains, VS Code,** mainstream
- **Cost:** $10–30/user/month
- **Best for:** Large organizations, standardization

Terminal-Native Tools Deep Dive

CLAUDE CODE
(Anthropic, 200K context)

- **No GUI required** (works over SSH!)
- Can execute bash, edit files, search codebases
- **MCP tool discovery** (integrate external tools)
- **Cost:** Pay per token (~$1–5/session)
- **Best for:** Backend work, scripting, one-offs

GEMINI CLI
(Google, 1M context)

- **FREE** with Gemini 2.5 Pro (GAME CHANGER)
- **Conductor extension** (automated multi-step)
- **Terminal-first,** bash-integrated
- **Cost:** FREE
- **Best for:** Cost-conscious teams, huge codebases

AIDER
(Open source, 100–200K context)

- **Model-agnostic** (Claude, GPT, Gemini, local)
- **Command-line** git-aware editor
- **Cost:** FREE (open source)
- **Cost:** FREE (open source)
- **Best for:** Open-source projects, local workflows

Rapid Prototyping Platforms

BOLT.NEW
(Browser, full-stack)

- **Generate:** frontend + backend + database
- **Deploy:** directly to web
- **Cost:** FREE (+ hosting)
- **Time to live demo:** 5 minutes
- **Best for:** MVPs, hackathons, learning

V0 BY VERCEL
(React + Vercel)

- **Next.js** component generation
- **Vercel** hosting built-in
- **Cost:** Freemium
- **Best for:** React startups, Vercel ecosystem

LOVABLE
(React, clean exports)

- **Exportable** projects
- **Full-stack** capable
- **Cost:** Freemium
- **Best for:** Learning, clean code

REPLIT AGENT
(Cloud IDE)

- **Integrated:** code + terminal + deploy
- **Collaborative**
- **Cost:** Freemium + paid
- **Best for:** Teaching, teams

Tool Selection Matrix

QUICK SELECTION GUIDE

Task: Boilerplate API	→ Cursor/Windsurf
Task: Rapid full-stack MVP	→ Bolt.new
Task: Large refactoring	→ Windsurf/ Claude Code
Task: Backend/scripting	→ Claude Code
Task: React components	→ v0 or Lovable
Task: Open-source contrib	→ aider
Task: Cost-sensitive, huge codebase	→ Gemini CLI
Task: Enterprise, controls	→ Copilot

GOLDEN PRINCIPLE:

"Tools change every 3 months, disciplines last decades"

Learn the discipline (prompting, context engineering, validation)
And you can use ANY tool.

This matrix should be printed and brought to every engineering decision.

Prompt Engineering for Code

Three Evolution Stages:

STAGE 1: Vague prompts

"Make a website"

Result: Inconsistent output, high iteration

STAGE 2: Structured prompts

"Create a React todo app with these requirements:

Scope: CRUD, localStorage, dark mode;
Non-Goals: Backend, auth, real-time;
Constraints: <5 KB gzipped, zero external deps"

Result: Consistent, high-quality output

STAGE 3: Context engineering

[Provide codebase structure]
[Provide style examples]
[Provide constraint files]
[Provide test cases]
Then: "Implement logout endpoint"

Result: Near-perfect output

KEY INSIGHT:
Your job is not to ask, but to **ORGANIZE INFORMATION**
So the AI can't be confused.

Homework

HOMEWORK (due next week):

Addy Osmani's "My LLM Coding Workflow Going Into 2026"

BY NEXT WEEK:

You'll have hands-on experience with multiple tools.

You'll understand which fits which job.